

MICROCURRÍCULO

Estructuras de Datos

1. INFORMACIÓN GENERAL

PROGRAMA	Ciencia de Datos
ÁREA	Computación y Programación
ASIGNATURA	Estructuras de Datos
CRÉDITOS	3
SEMESTRE	III
HORAS PRESENCIA- LES	96
HORAS DE TRABA- JO AUTÓNOMO	96
PROFESORES	Profesor 1 – arley.torres@uexternado.edu.co Profesor 2 – german.combariza@uexternado.edu.co

2. PRESENTACIÓN

La asignatura **Estructuras de Datos** constituye uno de los pilares fundamentales del eje computacional del programa. Su propósito es desarrollar en el estudiante la capacidad de *modelar, organizar y manipular información de manera eficiente*, comprendiendo cómo las representaciones internas y las decisiones de diseño influyen directamente en el comportamiento, la escalabilidad y la claridad de las soluciones computacionales. El curso ofrece las bases conceptuales y técnicas que permiten diseñar software robusto y algoritmos eficientes, competencias esenciales en la formación de un científico de datos.

Ubicada en el **tercer semestre**, esta asignatura amplía y formaliza los conocimientos adquiridos en Programación 1 y Programación 2. A partir de los mecanismos de abstracción, modularidad, recursión y programación orientada a objetos explorados previamente, el curso introduce estructuras lineales, jerárquicas y asociativas —arreglos, pilas, colas, listas enlazadas, árboles, heaps, tablas hash, mapas y grafos— estudiando sus modelos de operación, su representación en memoria y los costos asociados a cada operación. Todas las implementaciones se desarrollan en **Python**. Esta comprensión estructural es indispensable para diseñar algoritmos eficientes y para enfrentar los desafíos propios del análisis de datos a gran escala.

La formación se articula alrededor de tres ejes: el **análisis algorítmico riguroso**, la **comprensión de modelos de representación** y el **diseño fundamentado de estructuras**. A través de ellos, el estudiante aprende a evaluar alternativas, comparar comportamientos, detectar degradaciones, justificar la elección de una estructura y anticipar el impacto que tienen las decisiones de diseño sobre el rendimiento de un programa. Este proceso de razonamiento constituye la base para cursos como *Bases de Datos*, *Big Data*, *Machine Learning* y *MLOps*, donde la eficiencia estructural es un requisito indispensable.

El curso integra además espacios prácticos de laboratorio y dos proyectos, en los que el estudiante implementa estructuras desde cero, depura comportamientos, experimenta con diferentes representaciones y desarrolla soluciones completas que combinan varias estructuras en un mismo contexto. Estos espacios fortalecen la comprensión operativa de los modelos estudiados y fomentan el criterio profesional para diseñar sistemas organizados y escalables.

En conjunto, Estructuras de Datos busca que el estudiante desarrolle un pensamiento estructural maduro, caracterizado por la capacidad de analizar problemas complejos, seleccionar representaciones adecuadas, justificar decisiones de diseño y comprender la relación íntima entre modelo de datos, eficiencia algorítmica y calidad de la solución computacional. La asignatura constituye el puente hacia los cursos superiores del programa y un componente esencial en la formación profesional en Ciencia de Datos.

3. CONTINUIDAD CURRICULAR

Estructuras de Datos se ubica en el tercer semestre del eje computacional y constituye la progresión natural de **Programación 1** y **Programación 2**. Mientras estos cursos introducen el pensamiento algorítmico, la modularidad, la recursión, los mecanismos avanzados del lenguaje y los fundamentos de la programación orientada a objetos, esta asignatura se enfoca en comprender cómo se almacena, organiza y manipula la información de forma eficiente.

En el marco del plan de estudios, el curso desarrolla competencias esenciales para asignaturas como **Bases de Datos**, **Big Data**, **Machine Learning 1**, **Machine Learning 2**, **Visualización 1**, **MLOps** y **Deep Learning**. Estructuras de Datos proporciona el conocimiento técnico que permite seleccionar representaciones adecuadas, modelar conjuntos de información complejos, analizar costos computacionales y entender cómo las decisiones de diseño afectan el rendimiento de los algoritmos y sistemas utilizados en Ciencia de Datos. En particular, la comprensión de árboles de búsqueda y hashing sostiene directamente el estudio de los índices B-Tree y hash en Bases de Datos.

La asignatura se articula de manera directa con los fundamentos de **Matemáticas Discretas**, la lógica formal de **Cálculo II** y el razonamiento estructural de **Álgebra Lineal**. Conceptos como relaciones, funciones, grafos, crecimiento asintótico y propiedades de estructuras numéricas encuentran aquí su expresión computacional mediante pilas, colas, listas enlazadas, árboles, heaps, tablas hash, mapas y grafos.

Estructuras de Datos prepara al estudiante para enfrentar problemáticas reales de Ciencia de

Datos: manipulación a gran escala, optimización de consultas, diseño de modelos eficientes de almacenamiento, selección de estructuras para procesamiento masivo y comprensión de cómo algoritmos fundamentales dependen críticamente de las estructuras subyacentes.

En términos formativos, la asignatura consolida la transición entre el diseño avanzado introducido en Programación 2 y el razonamiento técnico que requiere el resto del eje computacional. El estudiante fortalece habilidades para analizar costos, diagnosticar degradaciones, comparar alternativas estructurales y justificar decisiones de diseño con base en criterios formales de eficiencia. Estas capacidades son cruciales para avanzar con solvencia hacia los cursos superiores del programa y para abordar los desafíos propios de la Ciencia de Datos contemporánea.

4. COMPETENCIAS

1. Competencia cognitiva. Orienta al estudiante hacia la comprensión profunda de las estructuras de datos, sus representaciones internas y el análisis formal de la eficiencia algorítmica. En este curso implica:

- Analizar problemas computacionales y seleccionar la estructura de datos adecuada —arreglos, pilas, colas, listas enlazadas, árboles, heaps, tablas hash, mapas o grafos— justificando la elección a partir de su modelo de operación y su costo.
- Comprender la organización interna de cada estructura, sus invariantes, sus operaciones fundamentales y su impacto en el rendimiento, utilizando notación asintótica, conteo de operaciones y recurrencias para evaluar eficiencia temporal y espacial.
- Interpretar, depurar e implementar estructuras complejas, identificando casos borde, degradaciones, violaciones de invariantes y oportunidades de mejora en la representación o en las operaciones.

2. Competencia comunicativa. Se centra en la capacidad para explicar, documentar y argumentar decisiones de diseño estructural con claridad y precisión técnica. En este curso implica:

- Explicar en lenguaje especializado el comportamiento de estructuras y algoritmos —recorridos, heapify, resolución de colisiones, BFS/DFS, caminos mínimos— justificando la selección de representaciones y operaciones.
- Documentar implementaciones y módulos describiendo la estructura utilizada, sus invariantes, parámetros, efectos, restricciones y supuestos algorítmicos, siguiendo buenas prácticas de organización y claridad.

3. Competencia investigativa. Comprende la habilidad para explorar, experimentar y validar ideas mediante la observación estructural, la contrastación de soluciones y el análisis crítico del comportamiento algorítmico. En este curso implica:

- Experimentar con distintas representaciones y estrategias —listas vs. arreglos, hashing con encadenamiento vs. direccionamiento abierto, árboles de búsqueda equilibrados vs. degradados, BFS vs. DFS— evaluando sus efectos en desempeño, claridad y escalabilidad.
- Probar casos particulares, construir contraejemplos, analizar trazas de ejecución, detectar degradaciones (por ejemplo, colisiones excesivas o árboles de búsqueda desbalanceados) y depurar implementaciones de forma rigurosa.
- Sustentar conclusiones sobre el diseño y comportamiento de estructuras y algoritmos, explicando con claridad los principios que justifican su corrección, eficiencia, robustez y pertinencia dentro de un problema real.

5. RESULTADOS DE APRENDIZAJE

RESULTADOS DE LA COMPETENCIA COGNITIVA

- Analizar problemas computacionales y seleccionar estructuras de datos adecuadas —arreglos, pilas, colas, listas enlazadas, árboles, heaps, tablas hash, mapas y grafos— justificando la elección a partir de su modelo de operación y de su complejidad temporal y espacial.
- Comprender y aplicar los principios fundamentales de diseño y análisis de estructuras de datos, utilizando notación asintótica, conteo de operaciones y recurrencias para evaluar el costo de las operaciones básicas y su comportamiento frente al crecimiento del tamaño de la entrada.
- Interpretar, evaluar e implementar estructuras y algoritmos asociados (recorridos, heapify, resolución de colisiones, recorridos en grafos, caminos mínimos), identificando errores lógicos, degradaciones de rendimiento, violaciones de invariantes y oportunidades de optimización.

RESULTADOS DE LA COMPETENCIA COMUNICATIVA

- Explicar con claridad y precisión el funcionamiento de estructuras de datos y algoritmos asociados —por ejemplo, cómo la forma de un árbol de búsqueda afecta su eficiencia, cómo opera una cola de prioridad basada en heaps, cómo se resuelven colisiones en una tabla hash o cómo progresa un recorrido BFS/DFS en un grafo.
- Documentar implementaciones de estructuras y módulos describiendo su propósito, las operaciones disponibles, sus invariantes, las precondiciones y postcondiciones relevantes, así como los costos esperados de uso en distintos escenarios.
- Comunicar de manera técnica y estructurada las decisiones de diseño tomadas al elegir o implementar una estructura de datos, justificándolas en términos de eficiencia, claridad, escalabilidad y pertinencia frente al problema planteado.

RESULTADOS DE LA COMPETENCIA INVESTIGATIVA

- Experimentar con distintas representaciones y variantes de estructuras de datos —por ejemplo, diferentes estrategias de hashing, implementaciones alternativas de listas o colas de prioridad— comparando su comportamiento mediante pruebas controladas y análisis de casos extremos.
- Formular y verificar hipótesis sobre el impacto de ciertas decisiones de diseño (tipo de representación, forma del árbol, función hash, forma de recorrido) en la eficiencia y estabilidad de las estructuras, utilizando mediciones empíricas, trazas de ejecución y razonamiento formal.
- Sustentar conclusiones sobre la corrección, eficiencia y adecuación de estructuras y algoritmos en contextos de Ciencia de Datos, integrando evidencia experimental, argumentos teóricos y criterios de diseño estructural sólido.

6. METODOLOGÍA

La metodología de **Estructuras de Datos** se fundamenta en un enfoque activo, analítico y práctico que privilegia la comprensión profunda de las representaciones internas, los modelos de operación y los costos asociados a cada estructura. El curso busca que el estudiante desarrolle criterio estructural, pueda anticipar el comportamiento de un algoritmo según la estructura utilizada, detecte degradaciones y justifique decisiones de diseño con base en evidencia conceptual y empírica.

La propuesta pedagógica se articula con el modelo académico de la Facultad, orientado al fortalecimiento del razonamiento lógico, la argumentación técnica y el diseño sistemático de soluciones. En este sentido, el curso profundiza en estructuras lineales, jerárquicas, asociativas y de grafos —arreglos, pilas, colas, listas enlazadas, árboles, heaps, tablas hash, mapas, BFS/DFS y caminos mínimos— promoviendo la lectura crítica de implementaciones, la depuración estructural y el análisis comparativo de alternativas. Estos elementos constituyen la base para cursos como *Bases de Datos*, *Big Data*, *Machine Learning 1*, *Visualización 1* y *MLOps*.

Las clases se desarrollan de manera **sincrónica**, complementadas con espacios **asincrónicos** que permiten reforzar la práctica, implementar estructuras completas, medir tiempos de ejecución, diseñar funciones hash y experimentar con distintas representaciones de grafos. Los recursos asincrónicos incluyen notebooks interactivos, ejercicios graduados, análisis de recorridos, actividades de depuración y práctica dirigida. Las implementaciones se desarrollan en Python, y los laboratorios y proyectos se entregan mediante Git y GitHub.

Las sesiones de clase se organizan en los siguientes momentos:

1. **Contextualización conceptual.** El docente presenta el propósito de la estructura a estudiar —pila, cola, lista enlazada, árbol, heap, tabla hash o grafo— explicando el problema computacional que resuelve, su modelo de acceso, sus invariantes y su rol dentro del diseño de algoritmos eficientes. Se discute la relevancia de la estructura frente al tamaño de la entrada y el tipo de operación dominante.

2. **Exposición breve y demostrativa.** Se introduce la estructura mediante explicaciones concisas, ejemplos guiados y demostraciones en vivo (*live coding*). El docente ilustra cómo se comporta cada operación (insertar, eliminar, buscar, recorrer, reordenar, resolver colisiones) utilizando diagramas, trazas y simulaciones. El énfasis está en comprender el *cómo* y el *por qué* del comportamiento, no en memorizar implementaciones.
3. **Discusión, análisis y depuración.** Se estudian implementaciones alternativas, se comparan operaciones, se identifican errores lógicos, se analizan casos borde y se rastrea el flujo de ejecución de algoritmos como heapify, cadenas de colisiones o recorridos BFS/DFS. El estudiante aprende a justificar la estructura seleccionada y a explicar las diferencias de eficiencia.
4. **Talleres, ejercicios aplicados y trabajo autónomo.** Se desarrollan actividades prácticas orientadas a integrar múltiples estructuras: implementación completa de estructuras lineales, diseño de árboles y heaps, construcción de tablas hash funcionales, programación de recorridos en grafos y desarrollo de proyectos que combinan varias estructuras. Estas prácticas fortalecen el criterio estructural, la autonomía y la capacidad de producir soluciones eficientes, escalables y claras.

7. EVALUACIÓN

La evaluación del curso de Estructuras de Datos se concibe como un proceso continuo y formativo que acompaña todo el semestre, orientado a valorar la comprensión conceptual, la capacidad de análisis algorítmico y la habilidad para implementar, comparar y justificar estructuras de datos fundamentales. La evaluación no se limita a verificar la corrección sintáctica del código, sino que busca evidenciar el razonamiento que sustenta el diseño, la eficiencia y la organización de cada solución.

Los instrumentos de evaluación permiten medir las competencias cognitivas, aplicadas y comunicativas desarrolladas en el curso:

- **Laboratorios.** Actividades prácticas orientadas a la implementación, análisis y comparación de estructuras de datos y algoritmos asociados. Evalúan el dominio técnico, la comprensión del flujo interno y la capacidad para depurar y justificar soluciones.
- **Proyectos.** Dos proyectos distribuidos a lo largo del semestre, centrados en el diseño, implementación y análisis de estructuras de datos —lineales, jerárquicas y asociativas— y sus aplicaciones a la Ciencia de Datos. Permiten evaluar el criterio algorítmico, la organización del código, la claridad del diseño y la capacidad para resolver problemas de mayor escala.
- **Parciales.** Dos evaluaciones sumativas que valoran la comprensión de los conceptos fundamentales: análisis de algoritmos, recursión, estructuras lineales, árboles, hashing, heaps, mapas y recorridos de grafos.

- **Examen final.** Evaluación integradora que recoge los aprendizajes estructurales del curso, enfocándose en el análisis, diseño y justificación de estructuras de datos complejas y sus algoritmos asociados.

El docente seleccionará los instrumentos específicos en cada corte evaluativo, garantizando coherencia con los objetivos del curso y con el nivel de profundidad esperado en cada bloque temático.

PORCENTAJES DE EVALUACIÓN

- Parcial 1: 25 %
- Parcial 2: 25 %
- Laboratorios: 10 %
- Proyectos (2 en total): 15 %
- Examen final: 25 %

Cada uno de estos componentes contará con su rúbrica correspondiente, lo que permitirá al estudiante comprender los criterios de evaluación, identificar fortalezas y reconocer oportunidades de mejora en su desarrollo dentro del eje computacional del programa.

FALLAS Y LINEAMIENTOS INSTITUCIONALES

Un estudiante reprueba si supera el 20 % de inasistencias. Esto equivale a 10 fallas (20 horas). El máximo permitido para mantenerse en el curso es de 9 fallas (18 horas).

8. TEMÁTICAS O CONTENIDOS

Nota (instancia 2026-2). La programación se ajusta al calendario oficial del semestre. De la guía docente se instancian 42 sesiones temáticas; no se dictan las sesiones 43 a 48. El miércoles 21 de octubre corresponde al Evento de Investigación y se desarrolla como taller, sin tema nuevo. El examen final se realiza en la semana de exámenes (17 al 20 de noviembre).

Semana	Sesión	Fecha	Contenido
1	Festivo	20 jul	
	1	22 jul	Análisis de algoritmos: notación asintótica, crecimiento y órdenes de magnitud

Semana	Sesión	Fecha	Contenido
	2	24 jul	Análisis de algoritmos: mejores, peores y promedios casos
2	3	27 jul	Análisis de algoritmos: conteo de operaciones y comparaciones
	4	29 jul	Laboratorio 1: Análisis de algoritmos
	5	31 jul	Recursión I: diseño, casos base y profundidad
3	6	3 ago	Recursión II: trazas, árboles de recursión y análisis del costo
	7	5 ago	Recursión III: recurrencias, eficiencia y patrones clásicos
	Festivo	7 ago	
4	8	10 ago	Laboratorio 2: Recursión
	9	12 ago	Arreglos: operaciones básicas, acceso y modificación (enunciado Proyecto 1)
	10	14 ago	Arreglos: operaciones básicas, acceso y modificación
5	Festivo	17 ago	
	11	19 ago	Pilas: concepto, operaciones e implementación
	12	21 ago	Colas: modelos FIFO, operaciones y variaciones
6	13	24 ago	Laboratorio 3: Arreglos, pilas y colas
	14	26 ago	Repaso (preparación Parcial 1)
	15	28 ago	Parcial 1
7	16	31 ago	Listas enlazadas I: nodos, referencias, inserción y eliminación
	17	2 sep	Listas enlazadas II: listas circulares y dobles
	18	4 sep	Listas enlazadas III: variantes, aplicaciones y análisis de costo
8	19	7 sep	Laboratorio 4: Listas enlazadas
	20	9 sep	Árboles binarios: estructura, propiedades y recorridos
	21	11 sep	Árboles binarios de búsqueda (BST): inserción, búsqueda y eliminación
9	22	14 sep	BST: eliminación, efecto de la forma del árbol y recorrido ordenado
	23	16 sep	Laboratorio 5: Árboles binarios y BST
	24	18 sep	Sustentación del Proyecto 1 y enunciado del Proyecto 2

Semana	Sesión	Fecha	Contenido
10	25	21 sep	Colas de prioridad: concepto, operaciones y aplicaciones
	26	23 sep	Heaps I: estructura, heapify y extracción del mínimo/máximo
	27	25 sep	Heaps II: construcción, heapsort y aplicaciones
11	28	28 sep	Laboratorio 6: Heaps y colas de prioridad
	29	30 sep	Tablas hash: funciones hash, colisiones y estrategias de resolución
	30	2 oct	Tablas hash: análisis, eficiencia y casos prácticos
<i>Semana de receso: 5 al 10 de octubre — sin clases</i>			
12	Festivo	12 oct	
	31	14 oct	Laboratorio 7: Tablas hash
	32	16 oct	Repaso (preparación Parcial 2)
13	33	19 oct	Parcial 2
	Evento	21 oct	Taller — Evento de Investigación (sin tema nuevo)
	34	23 oct	Retroalimentación del Parcial 2
14	35	26 oct	Introducción a la teoría de grafos
	36	28 oct	Grafos: representaciones (listas de adyacencia y matrices)
	37	30 oct	Grafos: recorridos BFS y DFS — análisis y aplicaciones
15	Festivo	2 nov	
	38	4 nov	Grafos: caminos mínimos — algoritmo de Dijkstra
	39	6 nov	Grafos: aplicaciones en problemas reales de Ciencia de Datos
16	40	9 nov	Laboratorio 8: Grafos
	41	11 nov	Sustentación del Proyecto 2
	42	13 nov	Taller integrador aplicado a Ciencia de Datos
Examen final (17 al 20 de noviembre)			

9. REFERENCIAS

BIBLIOGRAFÍA BÁSICA

- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley. Texto central del curso: desarrolla estructuras lineales, árboles, colas de prioridad, heaps, hashing, grafos y análisis algorítmico con un enfoque riguroso y en Python.
- Miller, B. N., & Ranum, D. L. (2011). *Problem Solving with Algorithms and Data Structures Using Python* (2nd ed.). Franklin, Beedle & Associates. Recurso pedagógico que combina explicación conceptual, implementación en Python y ejercicios prácticos sobre recursión, listas, árboles, hashing y grafos.
- Bhargava, A. (2016). *Grokking Algorithms: An Illustrated Guide for Programmers and Other Curious People*. Manning Publications. Libro ilustrado e intuitivo que apoya la comprensión de los fundamentos algorítmicos mediante ejemplos visuales de recorridos, recursión, colas de prioridad, hashing y grafos.

BIBLIOGRAFÍA COMPLEMENTARIA

- Cormen, T., Leiserson, C., Rivest, R., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Referencia clásica de análisis algorítmico que complementa los temas de árboles, grafos, hashing y estructuras avanzadas.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. Enfoque claro y visual del diseño de algoritmos y estructuras de datos, con especial fortaleza en grafos y estructuras asociativas.
- Goodrich, M. T., & Tamassia, R. (2002). *Algorithm Design: Foundations, Analysis, and Internet Examples*. Wiley. Referencia sólida sobre diseño y análisis de algoritmos, útil para fortalecer la comprensión de eficiencia, notación asintótica y recorridos.